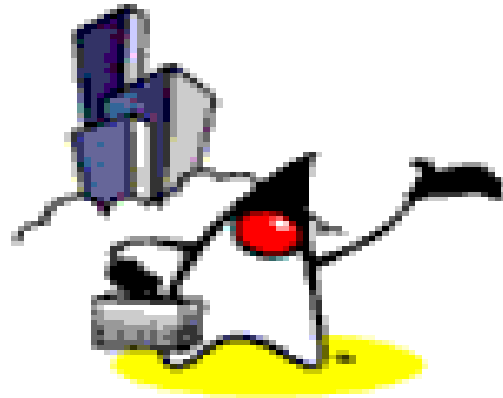


String Class



Topics

- *String* Class
- *StringBuffer* Class
- *StringBuilder* class



String Class

The *String* Class

- Definition:
 - Represents combinations of character literals
 - Using Java, strings can be represented using:
 - Array of characters
 - The *String* class
- Constructors of String class
 - 11 constructors

The *String* Class: Constructors

```
1 class StringConstructorsDemo {
2     public static void main(String args[]) {
3         String s1 = new String();           //empty string
4         char chars[] = { 'h', 'e', 'l', 'l', 'o' };
5         String s2 = new String(chars);      //s2="hello";
6         byte bytes[] = { 'w', 'o', 'r', 'l', 'd' };
7         String s3 = new String(bytes);      //s3="world"
8         String s4 = new String(chars, 1, 3);
9         String s5 = new String(s2);
10        String s6 = s2;
11    //continued
```

The *String* Class: Constructors

```
12      System.out.println(s1) ;
13      System.out.println(s2) ;
14      System.out.println(s3) ;
15      System.out.println(s4) ;
16      System.out.println(s5) ;
17      System.out.println(s6) ;
18  }
19 }
```

The *String* Class: Methods

String Methods
<pre>public char charAt(int index)</pre>
Returns the character located in the specified <i>index</i> .
<pre>public int compareTo(String anotherString)</pre>
Compares this string with the specified parameter. Returns a negative value if this string comes lexicographically before the other string, 0 if both of the strings have the same value and a positive value if this string comes after the other string lexicographically.
<pre>public int compareToIgnoreCase(String str)</pre>
Like <code>compareTo</code> but ignores the case used in this string and the specified string.
<pre>public boolean equals(Object anObject)</pre>
Returns true if this string has the same sequence of characters as that of the <i>Object</i> specified, which should be a <i>String</i> object. Otherwise, if the specified parameter is not a <i>String</i> object or if it doesn't match the sequence of symbols in this string, the method will return false.
<pre>public boolean equalsIgnoreCase(String anotherString)</pre>
Like <code>equals</code> but ignores the case used in this string and the specified string.
<pre>public void getChars(int srcBegin, int srcEnd, char[] dst, int dstBegin)</pre>
Gets the characters from this string starting at the <i>srcBegin</i> index up to the <i>srcEnd</i> index and copies these characters to the <i>dst</i> array starting at the <i>dstBegin</i> index.

The *String* Class: Methods

String Methods
<code>public int length()</code>
Returns the length of this string.
<code>public String replace(char oldChar, char newChar)</code>
Returns the string wherein all occurrences of the <i>oldChar</i> in this string is replaced with <i>newChar</i> .
<code>public String substring(int beginIndex, int endIndex)</code>
Returns the substring of this string starting at the specified <i>beginIndex</i> index up to the <i>endIndex</i> index.
<code>public char[] toCharArray()</code>
Returns the character array equivalent of this string.
<code>public String trim()</code>
Returns a modified copy of this string wherein the leading and trailing white space are removed.
<code>public static String valueOf(-)</code>
Takes in a simple data type such as boolean, integer or character, or it takes in an object as a parameter and returns the <i>String</i> equivalent of the specified parameter.

The *String* Class: Example

```
1 class StringDemo {
2     public static void main(String args[]) {
3         String name = "Jonathan";
4         System.out.println("name: " + name);
5         System.out.println("3rd character of name: " +
6                             name.charAt(2));
7         /* character that first appears alphabetically
8            has lower unicode value */
9         System.out.println("Jonathan compared to Solomon: "
10                             + name.compareTo("Solomon"));
11         System.out.println("Solomon compared to Jonathan: "
12                             + "Solomon".compareTo("Jonathan"));
13 //continued...
```

The *String* Class: Example

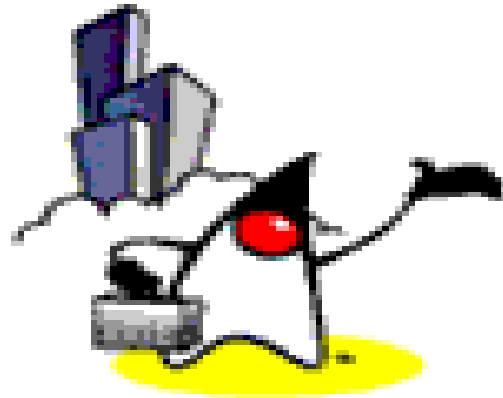
```
14      /* 'J' has lower unicode value compared to 'j' */
15      System.out.println("Jonathan compared to jonathan: " +
16                          name.compareTo("jonathan"));
17      System.out.println("Jonathan compared to jonathan
18 (ignore case): " + name.compareToIgnoreCase("jonathan"));
19      System.out.println("Is Jonathan equal to Jonathan? " +
20                          name.equals("Jonathan"));
21      System.out.println("Is Jonathan equal to jonathan? " +
22                          name.equals("jonathan"));
23      System.out.println("Is Jonathan equal to jonathan
24 (ignore case)? " + name.equalsIgnoreCase("jonathan"));
25 //continued...
```

The *String* Class: Example

```
26     char charArr[] = "Hi XX".toCharArray();
27     /* Need to add 1 to the endSrc index of getChars */
28     "Jonathan".getChars(0, 2, charArr, 3);
29     System.out.print("getChars method: ");
30     System.out.println(charArr);
31     System.out.println("Length of name: " +
32                         name.length());
33     System.out.println("Replace a's with e's in name: " +
34                         name.replace('a', 'e'));
35     // The substring begins at the specified beginIndex and
36     // extends to the character at index endIndex - 1
37     System.out.println("A substring of name: " +
38                         name.substring(0, 2)); // Jo
39 //continued...
```

The *String* Class: Example

```
40      System.out.println("Trim \"  a b c d e f  \": \"\" +  
41                          \"  a b c d e f  \".trim() + "\"");  
42      System.out.println("String representation of boolean  
43                          expression 10>10: \" + String.valueOf(10>10));  
44      /* toString method is implicitly called in the println  
45         method*/  
46      System.out.println("String representation of boolean  
47                          expression 10<10: \" + (10<10));  
48      /* Note there's no change in the String object name  
49         even after applying all these methods because  
50         String is a final class. */  
51      System.out.println("name: \" + name);  
52  }  
53 }
```



StringBuffer Class

StringBuffer class

- String objects are not mutable
 - Once String object is created, it cannot be modified
 - The length and content of the String object cannot be changed
- StringBuffer objects are mutable
 - Similar to String object
 - The length and content may be changed
 - Result in better performance for String manipulation

The *StringBuffer* Class: Methods

StringBuffer Methods	
<code>public int capacity()</code>	
Returns the current capacity of this <i>StringBuffer</i> object.	
<code>public StringBuffer append(-)</code>	
Appends the string representation of the argument to this <i>StringBuffer</i> object. Takes in a single parameter which may be of these data types: <i>boolean</i> , <i>char</i> , <i>char []</i> , <i>double</i> , <i>float</i> , <i>int</i> , <i>long</i> , <i>Object</i> , <i>String</i> and <i>StringBuffer</i> . Still has another overloaded version.	
<code>public char charAt(int index)</code>	
Returns the character located in the specified <i>index</i> .	
<code>public void getChars(int srcBegin, int srcEnd, char[] dst, int dstBegin)</code>	
Gets the characters from this object starting at the <i>srcBegin</i> index up to the <i>srcEnd</i> index and copies these characters to the <i>dst</i> array starting at the <i>dstBegin</i> index.	
<code>public StringBuffer delete(int start, int end)</code>	
Deletes the characters within the specified range.	
<code>public StringBuffer insert(int offset, -)</code>	
Inserts the string representation of the second argument at the specified offset. An overloaded method. Possible data types for the second argument: <i>boolean</i> , <i>char</i> , <i>char []</i> , <i>double</i> , <i>float</i> , <i>int</i> , <i>long</i> , <i>Object</i> and <i>String</i> . Still has another overloaded version.	

The *StringBuffer* Class: Example

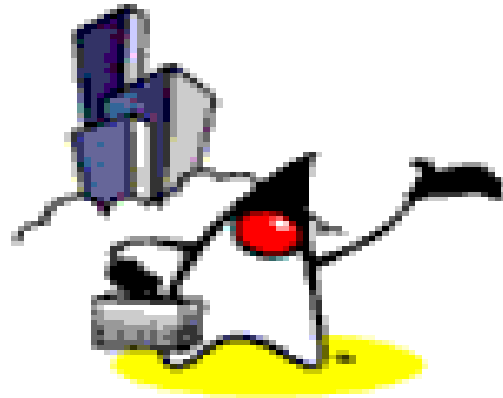
```
1 class StringBufferDemo {
2     public static void main(String args[]) {
3         StringBuffer sb = new StringBuffer("Jonathan");
4         System.out.println("sb = " + sb);
5         /* initial capacity is 16 */
6         System.out.println("capacity of sb: " + sb.capacity());
7         System.out.println("append \'O\' to sb: " +
8                             sb.append('O'));
9         System.out.println("sb = " + sb);
10        System.out.println("3rd character of sb: " +
11                            sb.charAt(2)); // n
12    } //continued...
```


The *StringBuffer* Class: Example

```
13      char charArr[] = "Hi XX".toCharArray();
14      /* Need to add 1 to the endSrc index of getChars */
15      sb.getChars(0, 2, charArr, 3);
16      System.out.print("getChars method: ");
17      System.out.println(charArr);
18      System.out.println("Insert \'jo\' at the 3rd cell: "
19                          + sb.insert(2, "jo"));
20      System.out.println("Delete \'jo\' at the 3rd cell: "
21                          + sb.delete(2,4));
22      System.out.println("length of sb: " + sb.length());
23  //continued
```

The *StringBuffer* Class: Example

```
24      System.out.println("replace: " +
25                          sb.replace(3, 9, " Ong"));
26      /* Need to add 1 to the endIndex parameter of
27         substring*/
28      System.out.println("substring (1st two characters): "
29                          + sb.substring(0, 3)); // Jon
30      System.out.println("implicit toString(): " + sb);
31  }
32 }
```



StringBuilder Class

StringBuilder

- StringBuilder class provides an API compatible with StringBuffer, but with no guarantee of synchronization
- This class is designed for use as a drop-in replacement for StringBuffer in places where the string buffer was being used by a single thread (as is generally the case)

Thank you!

